

7.1

判断下列命题的真伪。

(a) $2n = O(n)$ 。

真。取 $c = 2$, $n_0 = 1$, 则对所有 $n \geq n_0$,

$$2n \leq cn.$$

(b) $n^2 = O(n)$ 。

假。若成立, 则存在常数 c, n_0 , 使得对所有 $n \geq n_0$,

$$n^2 \leq cn.$$

等价于 $n \leq c$, 这不可能对任意大的 n 成立。

(c) $n^2 = O(n \log^2 n)$ 。

假。因为

$$\frac{n^2}{n \log^2 n} = \frac{n}{\log^2 n} \rightarrow \infty.$$

所以不存在常数 c 使得 $n^2 \leq cn \log^2 n$ 对充分大的 n 恒成立。

(d) $n \log n = O(n^2)$ 。

真。因为对充分大的 n , 有 $\log n \leq n$, 所以

$$n \log n \leq n^2.$$

(e) $3^n = 2^{O(n)}$ 。

真。因为

$$3^n = 2^{n \log_2 3}.$$

其中 $\log_2 3$ 是常数, 所以 $3^n = 2^{O(n)}$ 。

(f) $2^{2^n} = O(2^{2^n})$ 。

真。这是平凡成立的。取 $c = 1$, $n_0 = 1$ 即可。

7.2

判断下列命题的真伪。

(a) $n = o(2n)$ 。

假。因为

$$\lim_{n \rightarrow \infty} \frac{n}{2n} = \frac{1}{2} \neq 0.$$

(b) $2n = o(n^2)$ 。

真。因为

$$\lim_{n \rightarrow \infty} \frac{2n}{n^2} = \lim_{n \rightarrow \infty} \frac{2}{n} = 0.$$

(c) $2n = o(3^n)$ 。

真。因为指数函数增长快于线性函数，

$$\lim_{n \rightarrow \infty} \frac{2n}{3^n} = 0.$$

(d) $1 = o(n)$ 。

真。因为

$$\lim_{n \rightarrow \infty} \frac{1}{n} = 0.$$

(e) $n = o(\log n)$ 。

假。因为

$$\frac{n}{\log n} \rightarrow \infty,$$

而不是趋于 0。

(f) $1 = o(1/n)$ 。

假。因为

$$\frac{1}{1/n} = n \rightarrow \infty,$$

而不是趋于 0。

7.5

题目中的公式为

$$(x \vee y) \wedge (x \vee \bar{y}) \wedge (\bar{x} \vee y) \wedge (\bar{x} \vee \bar{y}).$$

该公式不可满足。

证明如下。前两个子句为

$$(x \vee y) \wedge (x \vee \bar{y}).$$

若 $x = \text{FALSE}$ ，则它们分别要求 $y = \text{TRUE}$ 和 $y = \text{FALSE}$ ，矛盾。因此必须有

$$x = \text{TRUE}.$$

后两个子句为

$$(\bar{x} \vee y) \wedge (\bar{x} \vee \bar{y}).$$

若 $x = \text{TRUE}$ ，则 $\bar{x} = \text{FALSE}$ ，这两个子句又分别要求

$$y = \text{TRUE} \quad \text{和} \quad y = \text{FALSE},$$

仍然矛盾。

所以不存在任何赋值能够同时满足四个子句，该公式不可满足。

7.9

证明

$$\text{TRIANGLE} \in \mathbf{P},$$

其中

$$\text{TRIANGLE} = \{\langle G \rangle \mid G \text{ 包含一个三角形}\}.$$

给出如下多项式时间算法。输入图 $G = (V, E)$ 。

1. 枚举所有三元组 $(u, v, w) \in V^3$ ，其中 u, v, w 两两不同。

2. 检查三条边

$$\{u, v\}, \quad \{v, w\}, \quad \{u, w\}$$

是否都属于 E 。

3. 如果存在这样的三元组，则接受；否则拒绝。

若 $|V| = n$ ，则不同三元组数量至多为 n^3 。若使用邻接矩阵表示图，则每次查边可在 $O(1)$ 时间内完成，总时间为

$$O(n^3).$$

因此该算法在多项式时间内运行，所以

$$\text{TRIANGLE} \in \mathbf{P}.$$

7.29

证明

$$\text{SET-SPLITTING}$$

是 NP 完全的。

题目定义为

$$\text{SET-SPLITTING} = \{\langle S, C \rangle \mid S \text{ 是有限集}, C = \{C_1, \dots, C_k\} \text{ 是 } S \text{ 的若干子集},$$

存在一种把 S 的元素染成红、蓝两色的方法，使得每个 C_i 中的元素不全为同一种颜色\}.

第一步：证明 $\text{Set-Splitting} \in \mathbf{NP}$

证书为 S 中每个元素的颜色。

验证机逐个检查每个集合 C_i ，判断其中是否至少有一个红色元素并且至少有一个蓝色元素。若所有 C_i 都满足该条件，则接受；否则拒绝。

该验证过程显然可在关于输入规模的多项式时间内完成。因此

$$\text{SET-SPLITTING} \in \mathbf{NP}.$$

第二步：证明 NP 难

从 3SAT 归约到 SET-SPLITTING。

设输入为一个 3cnf 公式

$$\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

变量为

$$x_1, x_2, \dots, x_n.$$

每个子句 C_j 中包含三个文字，记为

$$C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3}).$$

构造集合

$$S = \{z\} \cup \{x_i, \bar{x}_i \mid 1 \leq i \leq n\}.$$

其中 z 是一个特殊元素。

接下来构造子集族 C 。

首先，对每个变量 x_i ，加入集合

$$\{x_i, \bar{x}_i\}.$$

这会强制 x_i 和 $\bar{x}_i$ 被染成不同颜色。

其次，对每个子句

$$C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3}),$$

加入集合

$$\{z, \ell_{j1}, \ell_{j2}, \ell_{j3}\}.$$

该构造显然可在多项式时间内完成。

下面证明

$$\varphi \text{ 可满足} \iff \langle S, C \rangle \in \text{SET-SPLITTING}.$$

充分性

若 φ 可满足，取一个满足赋值。

把特殊元素 z 染成蓝色。对每个文字元素，若该文字在该赋值下为真，则染成红色；若为假，则染成蓝色。

于是，对每个变量 x_i ，由于 x_i 与 $\bar{x}_i$ 恰好一个为真、一个为假，所以集合

$$\{x_i, \bar{x}_i\}$$

中恰好有一个红色元素和一个蓝色元素。

对每个子句 C_j ，因为它被满足，所以三个文字中至少有一个为真，即对应元素为红色。而集合

$$\{z, \ell_{j1}, \ell_{j2}, \ell_{j3}\}$$

中包含蓝色元素 z ，又至少包含一个红色文字元素，因此它不是单色集合。

所以得到一个合法的集合划分。

必要性

反过来, 假设 $\langle S, C \rangle$ 有合法的红蓝染色。

由于颜色名称可以整体交换, 不妨设 z 为蓝色。

由集合

$$\{x_i, \bar{x}_i\}$$

不是单色集合可知, 对每个 i , x_i 与 $\bar{x}_i$ 颜色不同。

定义布尔赋值如下:

$$x_i = \text{TRUE} \iff x_i \text{ 的颜色不同于 } z \text{ 的颜色.}$$

也就是说, 若 x_i 为红色, 则令 $x_i = \text{TRUE}$; 若 x_i 为蓝色, 则令 $x_i = \text{FALSE}$ 。

考虑任意子句

$$C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3}).$$

对应集合

$$\{z, \ell_{j1}, \ell_{j2}, \ell_{j3}\}$$

不是单色集合, 并且 z 为蓝色。因此, 三个文字元素中至少有一个不是蓝色, 即至少有一个为红色。根据赋值定义, 该红色文字为真。因此每个子句都被满足。

所以 φ 可满足。

综上,

$$3SAT \leq_p \text{SET-SPLITTING}.$$

因为 $3SAT$ 是 NP 完全的, 且 SET-SPLITTING 属于 **NP**, 所以

$$\text{SET-SPLITTING}$$

是 NP 完全的。

7.30

题目中的考试安排问题如下: 给定考试

$$F_1, \dots, F_k$$

以及学生清单

$$S_1, \dots, S_\ell,$$

每个学生选择了这些考试中的某个子集。问是否能把考试安排到 h 个时段中, 使得同一时段中没有某个学生同时需要参加两门考试。

证明该问题是 NP 完全的。

第一步：证明该问题属于 NP

证书为一个考试安排函数

$$f : \{F_1, \dots, F_k\} \rightarrow \{1, 2, \dots, h\},$$

其中 $f(F_i)$ 表示考试 F_i 被安排在哪个时段。

验证时，对每个学生清单 S_j ，检查其中任意两门考试是否被安排在同一时段。如果存在冲突，则拒绝；若所有学生清单均无冲突，则接受。

该检查最多需要枚举每个 S_j 中的考试对，显然可在多项式时间内完成。因此该问题属于 **NP**。

第二步：证明 NP 难

从 *3COLOR* 归约到该考试安排问题。

给定一个无向图

$$G = (V, E),$$

构造一个考试安排实例。

- 对每个顶点 $v \in V$ ，建立一门考试 F_v 。
- 对每条边 $\{u, v\} \in E$ ，建立一个学生清单

$$S_{\{u,v\}} = \{F_u, F_v\}.$$

也就是说，这个学生同时选择了与边的两个端点对应的两门考试。

- 令时段数

$$h = 3.$$

该构造显然可在多项式时间内完成。

下面证明

$$G \text{ 可 3-着色} \iff \text{对应考试安排实例有合法安排.}$$

若 G 有一个合法的 3-着色，则把三种颜色分别看作三个考试时段。对每个顶点 v ，将考试 F_v 安排到顶点 v 的颜色对应的时段。由于每条边的两个端点颜色不同，所以对每个学生清单

$$\{F_u, F_v\},$$

两门考试不会在同一时段。因此得到合法考试安排。

反过来，若考试安排实例有合法安排，则把每个顶点 v 的颜色定义为考试 F_v 所在的时段。由于每条边 $\{u, v\}$ 对应一个学生清单 $\{F_u, F_v\}$ ，合法安排要求 F_u 与 F_v 不在同一时段，所以相邻顶点颜色不同。因此得到 G 的一个合法 3-着色。

所以

$$3\text{COLOR} \leq_p \text{考试安排问题.}$$

由于 *3COLOR* 是 NP 完全的，且考试安排问题属于 **NP**，所以该考试安排问题是 NP 完全的。

7.36

证明：若

$$\mathbf{P} = \mathbf{NP},$$

则可以在多项式时间内将整数因子分解。

考虑如下判定语言：

$$L_{\text{fac}} = \{\langle N, a, b \rangle \mid \text{存在整数 } d, a \leq d \leq b, 1 < d < N, d \mid N\}.$$

首先，

$$L_{\text{fac}} \in \mathbf{NP}.$$

因为证书可以取为整数 d 。验证机只需检查

$$a \leq d \leq b, \quad 1 < d < N, \quad d \mid N,$$

这些检查都能在关于输入长度的多项式时间内完成。

若 $\mathbf{P} = \mathbf{NP}$ ，则

$$L_{\text{fac}} \in \mathbf{P}.$$

也就是说，我们可以在多项式时间内判断区间 $[a, b]$ 中是否存在 N 的非平凡因子。

下面利用该判定算法寻找因子。

对输入整数 N ，先查询

$$\langle N, 2, N-1 \rangle.$$

如果答案为否，则 N 没有非平凡因子，因此 N 是素数，输出 N 。

如果答案为是，则说明 N 有某个非平凡因子。接下来在区间 $[2, N-1]$ 上进行二分查找。设当前区间为 $[a, b]$ ，令

$$m = \left\lfloor \frac{a+b}{2} \right\rfloor.$$

查询

$$\langle N, a, m \rangle.$$

若答案为是，则因子存在于 $[a, m]$ 中，将区间缩小为 $[a, m]$ ；否则因子存在于 $[m+1, b]$ 中，将区间缩小为 $[m+1, b]$ 。

经过 $O(\log N)$ 次查询后，可以找到一个非平凡因子 d 。

然后递归分解

$$d \text{ 和 } N/d.$$

每次递归都把当前整数分解成两个更小的正整数。由于输入长度为 $\log N$ ，递归层数和输出的因子个数均至多为多项式数量级。每一步只调用多项式时间判定算法，并做多项式时间整数运算。

因此，在假设 $\mathbf{P} = \mathbf{NP}$ 的条件下，整数因子分解可以在多项式时间内完成。

7.39

题目定义了谜题语言

$$\text{PUZZLE} = \{\langle c_1, \dots, c_k \rangle \mid \text{每个 } c_i \text{ 代表一张卡片, 并且这个卡片集有解}\}.$$

证明

PUZZLE

是 NP 完全的。

第一步：证明 $\text{Puzzle} \in \text{NP}$

证书为每张卡片的放置方式。因为每张卡片只有两种可能的放置方式，所以证书可表示为一个长度为 k 的二进制串。

验证机根据证书摆放所有卡片，然后检查盒子底部的每个孔位是否都被至少一张在该位置上无孔的卡片覆盖。该检查只需扫描所有孔位和所有卡片，时间为输入规模的多项式。因此

$$\text{PUZZLE} \in \text{NP}.$$

第二步：证明 NP 难

从 3SAT 归约到 PUZZLE。

设输入为一个 3cnf 公式

$$\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_m,$$

变量为

$$x_1, x_2, \dots, x_n.$$

每个子句 C_j 是三个文字的析取。

构造一个谜题实例。盒子底部设置 m 个需要覆盖的孔位，分别对应 m 个子句

$$C_1, \dots, C_m.$$

对每个变量 x_i ，构造一张卡片。该卡片有两种放置方式，分别解释为

$$x_i = \text{TRUE} \quad \text{和} \quad x_i = \text{FALSE}.$$

卡片的孔位设计如下：

- 若选择 $x_i = \text{TRUE}$ 的放置方式，则该卡片在第 j 个孔位处为无孔位置，当且仅当文字 x_i 出现在子句 C_j 中。
- 若选择 $x_i = \text{FALSE}$ 的放置方式，则该卡片在第 j 个孔位处为无孔位置，当且仅当文字 $\bar{x}_i$ 出现在子句 C_j 中。
- 在其他位置，该卡片打孔，使其不能覆盖对应孔位。

直观地说，变量卡片在某种放置方式下，正好覆盖被该赋值满足的那些子句对应的孔位。

该构造的规模为 $O(nm)$ ，因此可以在多项式时间内完成。

下面证明

$$\varphi \text{ 可满足} \iff \text{构造出的谜题实例有解.}$$

充分性

若 φ 有满足赋值，则按照该赋值放置每张变量卡片：若 $x_i = \text{TRUE}$ ，选择卡片的真放置方式；若 $x_i = \text{FALSE}$ ，选择卡片的假放置方式。

由于每个子句 C_j 至少有一个文字为真，所以至少存在一张变量卡片在第 j 个孔位处无孔，从而覆盖该孔位。于是所有子句对应的孔位都被覆盖，谜题有解。

必要性

反过来，若谜题实例有解，则根据每张变量卡片的放置方式定义布尔赋值：若第 i 张卡片选择真放置方式，则令 $x_i = \text{TRUE}$ ；若选择假放置方式，则令 $x_i = \text{FALSE}$ 。

因为谜题有解，所以盒子底部每个孔位都被覆盖。考虑第 j 个孔位。它被某张变量卡片覆盖，说明在对应赋值下，某个出现在子句 C_j 中的文字为真。因此每个子句都至少有一个真文字，故 φ 被满足。

所以

$$3SAT \leq_p \text{PUZZLE}.$$

由于 $3SAT$ 是 NP 完全的，且 PUZZLE 属于 **NP**，因此

$$\text{PUZZLE}$$

是 NP 完全的。

7.40

题目定义

$$\text{MODEXP} = \{\langle a, b, c, p \rangle \mid a, b, c, p \text{ 是二进制整数, 且 } a^b \equiv c \pmod{p}\}.$$

证明

$$\text{MODEXP} \in \mathbf{P}.$$

关键是不能直接计算 a^b ，因为 a^b 可能有指数级长。应当始终在模 p 的意义下计算。

首先考虑 b 是 2 的幂的情形。若

$$b = 2^t,$$

则可以通过反复平方计算

$$a, a^2, a^4, a^8, \dots, a^{2^t}$$

在模 p 下的值。具体地，令

$$r_0 = a \bmod p,$$

并递推

$$r_{i+1} = r_i^2 \bmod p.$$

则

$$r_i \equiv a^{2^i} \pmod{p}.$$

由于 $t \leq |b|$ ，只需要多项式次模乘运算。

一般情形下，将 b 写成二进制：

$$b = \sum_{i=0}^{m-1} b_i 2^i, \quad b_i \in \{0, 1\}.$$

先用反复平方计算

$$r_i \equiv a^{2^i} \pmod{p}$$

对于所有 $0 \leq i < m$ 的值。

然后令

$$s = 1.$$

对每个满足 $b_i = 1$ 的位置，更新

$$s \leftarrow s \cdot r_i \bmod p.$$

最终得到

$$s \equiv \prod_{i:b_i=1} a^{2^i} = a^{\sum_{i:b_i=1} 2^i} = a^b \pmod{p}.$$

最后检查

$$s \equiv c \pmod{p}$$

是否成立。若成立则接受，否则拒绝。

整个过程中，所有中间数都小于 p ，其二进制长度至多为输入长度。算法只进行 $O(|b|)$ 次模乘和模平方运算，而整数加法、乘法、除法取模都可以在多项式时间内完成。因此该算法总运行时间为输入长度的多项式。

所以

$$\text{MODEXP} \in \mathbf{P}.$$